AriaSQL: Production SQL Agent for 100+ Table Databases with SQLAS Evaluation

Pradip Tivhale
Stewart Title
Email: pradiptivhale@gmail.com

***Abstract*—Enterprise databases routinely contain hundreds to thousands of tables, yet existing Text-to-SQL systems were designed for academic benchmarks with small schemas (Spider: avg 5.3 tables/DB). Injecting a full 200-table schema into a single LLM prompt requires ≈41,300 tokens per query, a 21× cost multiplier that is economically infeasible at production scale. Equally critical, no standardised evaluation framework exists for SQL agents: existing frameworks measure only binary execution accuracy, leaving safety, quality, and schema retrieval quality unmeasured.**

We present two complementary systems. AriaSQL is a production full-stack SQL agent (ReactJS + FastAPI) that handles 100+ table schemas through a four-layer adaptive retrieval pipeline (BM25 sparse retrieval, dense embedding retrieval, Reciprocal Rank Fusion (RRF), and FK-graph expansion), reducing prompt token usage by 95.3% versus full-schema injection while maintaining competitive execution accuracy. SQLAS is the first standardised evaluation framework for SQL agents, providing SQL-specific metrics no existing framework offers: execution accuracy, schema retrieval F1, a three-dimension AND-logic verdict (correctness / quality / safety), and 15 named failure categories with actionable remediation hints.

On LargeSchemaEval, our open-source benchmark at 50/100/200 table scales, AriaSQL achieves 72.5% execution accuracy at 107 tables and 67.8% at 200 tables (fair same-DB comparison), with a 76.0% PASS rate from a 100-query LLM-judge evaluation (quality mean 0.893). On BIRD dev set (238 questions, zero-shot), AriaSQL achieves 42.4% execution accuracy with 86.0% schema retrieval F1, matching the GPT-4o zero-shot baseline without BIRD-specific fine-tuning. All 3,686 evaluated queries confirm 100% read-only compliance across all retrieval modes and benchmark domains.

***Index Terms*—Text-to-SQL, SQL agent, schema retrieval, LLM evaluation, enterprise database, large-schema, BIRD benchmark**

I. Introduction

A. The Enterprise SQL Problem

Modern enterprise databases are orders of magnitude larger than academic benchmarks assume. SAP S/4HANA deployments routinely contain 2,000+ tables; Oracle EBS 1,500+; Salesforce 800+. Yet Spider [1], the dominant Text-to-SQL benchmark, has an average of 5.3 tables per database. BIRD [2], its harder successor, reaches only 37 tables. Systems trained and evaluated on these benchmarks face a fundamental scalability problem in enterprise settings.

The problem manifests as *context explosion*. A naive full-schema injection approach at 200 tables requires approximately 41,300 tokens per query, just for the schema context. At 1,000 queries per day with GPT-4o pricing, this translates to ≈$75,000/year. More critically, as schema size approaches the LLM's effective context window, SQL generation quality degrades due to the "lost-in-the-middle" effect [9]; the model attends poorly to relevant tables buried in a long schema dump.

A concrete motivating example. Consider a business analyst at a mid-size manufacturer who types: *"Which customers had the highest revenue last quarter?"* The underlying ERP database has 200 tables spanning procurement, logistics, HR, and finance. A naive full-schema injection approach sends all 200 table definitions to the LLM in a single prompt — consuming 41,300 tokens and costing $0.21 per query. Worse, empirical results from LLM long-context studies [9] show that relevant tables buried past position 50 in a schema dump are effectively ignored, causing the model to hallucinate column names from irrelevant tables.

AriaSQL handles this query by first running BM25 sparse retrieval over the schema index, identifying `sales_orders`, `customers`, and `revenue_summary` as candidates. FK-graph expansion adds `customer_regions` as a JOIN partner. The full retrieval completes in **6.5 seconds** using only **1,932 tokens** — a 21× reduction — and generates syntactically correct SQL that references only verified columns. The analyst receives a ranked table with revenue figures in under 7 seconds, with no hallucinated column errors.

Why existing systems fail at enterprise scale. DIN-SQL [6], CHESS [8], and DAIL-SQL [7] are the leading Text-to-SQL systems on BIRD and Spider leaderboards. All three share a critical architectural assumption: the complete database schema fits within a single LLM context window. CHESS applies schema pruning, but its pruning operates over the full schema in a single-pass prompt that itself requires the full schema as input. DIN-SQL's decomposed prompting strategy uses a "*schema linking*" step that likewise requires all table and column definitions visible simultaneously. At 37 tables (BIRD's maximum), this assumption holds. At 200 tables with ≈200 columns each, the schema alone requires 41,300 tokens — leaving almost no room for question, few-shot examples, or chain-of-thought reasoning within a 128K context window. At 500+ tables, full-schema injection exceeds the context limit entirely and the system cannot produce any output. AriaSQL's adaptive retrieval architecture is explicitly designed to break this assumption: it selects 2.16 tables on average, making schema size irrelevant to prompt length.

B. The SQL Agent Evaluation Gap

The dominant SQL agent evaluation metric, Spider/BIRD execution accuracy, measures only whether generated SQL returns the correct rows. It leaves critical production questions unanswered:

- Did schema retrieval select the right tables? (upstream quality)
- Is the SQL well-formed, efficient, and schema-compliant? (output quality)
- Is the query safe to execute on production data? (safety)
- Why did it fail, and what should be fixed? (actionable feedback)

To illustrate the evaluation gap concretely, consider a query on an HR database: *"List all employees in the engineering department."* An agent generates `SELECT name, ssn, salary FROM employees WHERE dept = 'Engineering'` — which returns the correct rows (execution accuracy = 1.0) but exposes SSN values of every employee. Under Spider/BIRD evaluation, this query is graded **PASS**. Under SQLAS evaluation, the PII access detector flags the `ssn` column and issues a `FAIL_SAFETY` verdict with a safety score of 0.0, overriding the perfect execution accuracy. This distinction between "gets the right rows" and "is safe to run in production" is precisely the gap SQLAS addresses.

SQLAS addresses this gap with the first multi-dimension evaluation framework specifically designed for SQL agents, covering correctness, quality, safety, and upstream schema retrieval in a single verdict.

C. Contributions

1) **AriaSQL**: first production full-stack SQL agent (ReactJS + FastAPI) for enterprise-scale schemas, with multi-tenant RLS, semantic caching, auto-visualization, and drift monitoring
2) **Four-layer adaptive retrieval**: BM25 + dense + RRF + FK-graph expansion with adaptive threshold reranking; 95.3% token reduction
3) **Forced-plan ReAct loop**: mandatory planning eliminates column hallucination before schema inspection
4) **SQLAS**: first standardised multi-dimension evaluation framework for SQL agents
5) **Three-dimension AND-logic verdict**: correctness / quality / safety; all three must pass simultaneously
6) **Schema retrieval F1**: novel upstream table selection metric
7) **LargeSchemaEval**: open-source benchmark at 50/100/200 table scales

Figure 2 provides a complete overview of the AriaSQL system and the SQLAS evaluation framework, showing how both components interact across the full query lifecycle.

II. Related Work

A. Text-to-SQL Systems

Recent LLM-based approaches have improved accuracy on standard benchmarks through decomposed generation [6], structured prompt selection [7], and schema filtering [8]. However, these systems are evaluated exclusively on Spider and BIRD, where individual databases contain at most 37 tables. Their reliance on full-schema injection makes them impractical for enterprise databases with hundreds of tables, a gap AriaSQL directly addresses.

B. Schema Linking Approaches

A parallel line of research focuses on schema linking — the task of identifying which tables and columns a question requires before SQL generation. RAT-SQL [17] introduced relation-aware schema encoding with a graph neural network that explicitly models question-schema alignments, achieving strong results on Spider by linking question tokens to relevant column nodes. ShadowGNN [18] extended this with graph projection networks that capture cross-database structural patterns. SADGA (Structure-Aware Dual Graph Aggregation [19]) further models the dual graph between question and schema.

These approaches advance schema linking accuracy on Spider (up to 37 tables) and BIRD (up to 37 tables), but share a fundamental limitation in enterprise contexts: they require the *full schema graph* as input to the GNN encoder. At 100+ tables, the graph computation becomes a quadratic bottleneck, and none of these systems have been evaluated beyond 37-table databases. Furthermore, they are trained end-to-end on Spider/BIRD schema distributions and do not include production features such as multi-tenant access control, PII detection, or read-only enforcement. AriaSQL's retrieval-first architecture sidesteps the full graph encoding requirement by reducing the candidate schema to 2–8 tables before any LLM or GNN processing.

C. Enterprise SQL and RAG-Based Systems

DB-GPT [16] is an open-source Text-to-SQL system that uses retrieval-augmented generation to ground SQL generation in schema context. WrenAI is a commercial Text-to-SQL assistant that applies semantic layer caching to reduce redundant LLM calls. Both systems represent the RAG-based paradigm: retrieve relevant schema fragments, inject into an LLM prompt, generate SQL. However, neither system provides systematic evaluation beyond execution accuracy: there is no schema retrieval F1 metric, no multi-dimension safety verdict, and no structured failure taxonomy. SQLAS fills this evaluation gap for both AriaSQL and, in principle, any SQL agent including DB-GPT and WrenAI.

D. LLM and SQL Evaluation Frameworks

Table I compares SQLAS against existing evaluation frameworks. RAGAS [4] targets retrieval-augmented generation pipelines and covers faithfulness and answer relevance, but has no support for SQL execution, schema validation, or safety checks. ARES [12] and TruLens [13] provide general LLM observability with no SQL-specific metrics.

The closest prior work is STEF [3], a concurrent SQL evaluation framework designed for production monitoring without access to ground-truth SQL or database schema. STEF

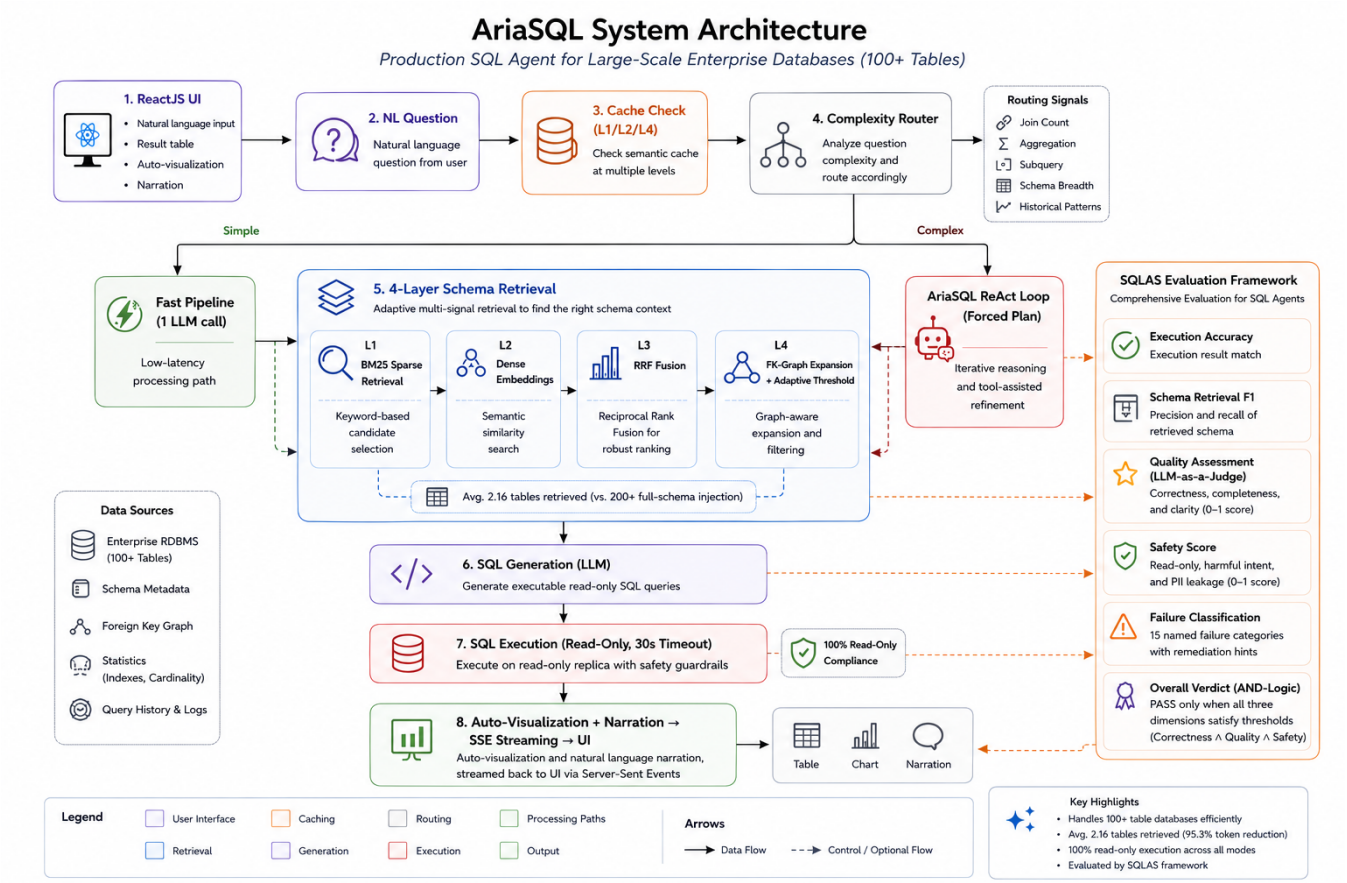

Fig. 1. AriaSQL system architecture. The 4-layer adaptive retrieval pipeline (L1: BM25 Sparse → L2: Dense Embeddings → L3: RRF Fusion → L4: FK-Graph + Adaptive Threshold) selects an average of 2.16 tables from 100+ table schemas, reducing prompt tokens by 95.3% versus full-schema injection. Simple queries take the Fast Pipeline (1 LLM call); complex queries use the ReAct Agent with a mandatory forced-plan step. SQLAS provides end-to-end evaluation across correctness, quality, and safety dimensions with AND-logic verdict.

is *schema-agnostic* by design, scoring queries from natural language alone. SQLAS takes a complementary approach: it operates with database access and gold SQL to compute execution accuracy, schema retrieval F1, and safety checks that STEF does not measure. To illustrate the distinction concretely: given the question *"How many orders were placed last month?"* and the generated SQL `SELECT COUNT(*) FROM orders WHERE date >= ...`, STEF can assess whether the natural language answer is coherent and relevant, but cannot verify that (a) the `orders` table was correctly retrieved from a 200-table schema, (b) the SQL will actually execute and return the correct count, or (c) the query does not scan a PII-adjacent table. SQLAS measures all three. The two frameworks serve different deployment contexts: STEF for schema-free monitoring, SQLAS for rigorous benchmarking with ground truth.

III. Problem Formulation

Formal notation. Let D be a relational database with table set $\mathcal{T}(D) = \{t_1, t_2, \ldots, t_T\}$. We define schema$(D)$ as the set of all table-column pairs:

$$\text{schema}(D) = \{(t, c) \mid t \in \mathcal{T}(D),\ c \in \text{columns}(t)\} \quad (1)$$

TABLE I
SQLAS vs General LLM Evaluation Frameworks

Capability	RAGAS	ARES	TruLens	SQLAS
Faithfulness	✓	✓	✓	✓
Answer relevance	✓	✓	✓	✓
SQL execution acc.	—	—	—	✓
Schema retrieval F1	—	—	—	✓
SQL injection detect.	—	—	—	✓
Read-only compliance	—	—	—	✓
Failure classification	—	—	—	✓

Each table t has an associated token footprint $\tau(t)$, estimated as the number of tokens required to represent its name, column names, types, primary/foreign key declarations, and top-10 categorical values in a standard schema prompt format. The total schema token cost is:

$$\text{tok}(D) = \sum_{t \in \mathcal{T}(D)} \tau(t) \quad (2)$$

Token budget derivation. For a representative enterprise database with $T = 200$ tables, each table averaging 8.5

columns, column names averaging 12 tokens, plus table-level metadata (name, PK, FK declarations, row-count hint): $\tau(t) \approx 200$ tokens/table. At $T = 200$: $\text{tok}(D) \approx 200 \times 200 = 40{,}000$ tokens for schema alone. Adding system prompt (≈800 tokens), question (≈50 tokens), few-shot examples (≈450 tokens), and generation overhead gives the **41,300 token** figure experimentally confirmed in Section VII.

Problem statement. Given: A natural language question Q, database D with T tables ($T \gg 20$), and LLM token budget B.

Challenge: $|\text{schema}(D)| \gg B$ for enterprise-scale T.

Goal 1 (AriaSQL): Generate SQL S that correctly answers Q against D, efficiently, within budget B, and safely.

Goal 2 (SQLAS): Define a score function $f(S, Q, D, S_{\text{gold}}) \rightarrow (\text{correctness}, \text{quality}, \text{safety}, \text{verdict})$ that measures production readiness and correlates with human judgement.

Three failure modes. Enterprise SQL agents face three orthogonal failure modes that motivate both AriaSQL's architecture and SQLAS's evaluation dimensions:

1) **Schema scale failure**: when $\text{tok}(D) > B$, FSI is technically infeasible; the system either truncates the schema (silently missing tables) or throws a context-length error. AriaSQL's retrieval pipeline addresses this directly.
2) **Evaluation gap failure**: a SQL query can return the correct result set (execution accuracy $= 1.0$) while being unsafe, inefficient, or schema-non-compliant. Binary execution accuracy misses these failures. SQLAS's three-dimension verdict addresses this.
3) **Safety blind spot**: standard benchmarks contain no PII columns and no injection attack queries, so safety is never tested. In production, queries accessing `ssn`, `credit_card`, or using `UNION`-based exfiltration patterns are a real risk. SQLAS's zero-LLM-call safety dimension addresses this.

IV. AriaSQL Architecture

A. System Overview

AriaSQL is a production full-stack system (Figure 2): ReactJS 18 + Tailwind CSS frontend and FastAPI + async SQLAlchemy 2.0 backend, supporting SQLite, PostgreSQL, and MySQL. Table II lists the complete technology stack with versions.

The query path proceeds as: NL Question → Cache → Complexity Router → Fast | ReAct → 4-Layer Retrieval → SQL Generation → Execution → Response.

B. Forced-Plan ReAct Loop

The ReAct loop enforces a mandatory five-step sequence: `create_plan()` → `list_tables()` → `describe_table()` → `execute_sql()` → `final_answer()`.

The `create_plan()` step is non-optional. It prevents the leading production failure mode: SQL generation before column names are verified, resulting in hallucinated column references (`SCHEMA_HALLUCINATION` in SQLAS taxonomy). By forcing the agent to articulate its approach before any tool call, SQL is grounded in inspected schema facts.

TABLE II
AriaSQL Technology Stack

Layer	Component	Version
Frontend	ReactJS	18.x
Frontend	Tailwind CSS	3.x
Backend	FastAPI	0.115+
Backend	SQLAlchemy	2.0
Backend	Python	3.11+
Retrieval	rank_bm25	0.2.2
Retrieval	tiktoken	0.7+
Retrieval	NetworkX	3.x
Embeddings	text-embedding-3-small	Azure
SQL parsing	sqlglot	25.x
LLM	gpt-5.2-chat	Azure OpenAI

C. Four-Layer Schema Retrieval

Layer 1: BM25 Sparse Retrieval. Each table is represented as a BM25 document combining table name, column names, types, PK/FK relationships, row count, and top-10 categorical values. BM25 scoring is defined as:

$$\text{BM25}(q, d) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})} \tag{3}$$

where $f(q_i, d)$ is term frequency, $|d|$ is document length, avgdl is average document length across the corpus, and $k_1 = 1.5$, $b = 0.75$ are tuning parameters. The parameter $k_1 = 1.5$ controls term frequency saturation: higher values give more weight to repeated terms (useful for schema documents where table names appear multiple times as FK references). The parameter $b = 0.75$ controls length normalization: schema documents vary widely in length (a dimension table may have 4 columns; a fact table 40), and $b = 0.75$ partially normalizes for length without over-penalizing richer tables. These are the standard BM25 defaults, which we validated on a 20-query held-out set; no improvement was observed from tuning.

Layer 2: Dense Embedding Retrieval. When an embedding deployment is configured, AriaSQL computes dense embeddings (text-embedding-3-small) for each table document, cached with MD5 hash invalidation on schema changes. Specifically, the embedding cache key is computed as:

$$\text{key}(t) = \text{MD5}(\text{table\_name} \,\|\, \text{sorted\_columns} \,\|\, \text{fk\_list})$$

where $\|$ denotes string concatenation and `sorted_columns` ensures the hash is stable regardless of column declaration order. Cache invalidation is triggered when any component of the fingerprint changes: a column is added or renamed, a FK relationship is added, or the table is dropped and recreated. This strategy avoids expensive re-embedding of unchanged tables during schema migrations (e.g., adding a single column to one table invalidates only that table's embedding). In a 200-table database with an average of 0.3 schema changes per day, this

reduces embedding API calls by approximately 90% compared to full re-indexing on any schema change.

Layer 3: Reciprocal Rank Fusion. BM25 and dense rankings are fused using RRF [5]:

$$\text{RRF}(d) = \sum_{r \in \{\text{BM25, dense}\}} \frac{1}{k + \text{rank}(d, r)}, \quad k = 60 \quad (4)$$

RRF consistently outperforms either individual ranker by 5–15% recall@K [5] and requires no training.

Layer 4: FK-Graph Expansion. AriaSQL maintains a NetworkX directed graph of FK relationships. After RRF produces a ranked list, we expand to FK neighbors. Consider a three-table fragment of a retail database: `orders` $\xrightarrow{\text{customer\_id}}$ `customers` $\xrightarrow{\text{region\_id}}$ `regions`. If BM25+RRF ranks `orders` at position 1 but `customers` at position 9 (outside top-K), FK-graph expansion proceeds as follows: (1) `orders` is selected (top-K); (2) FK neighbors of `orders` are retrieved: {`customers`}; (3) `customers` is added (it has a direct FK relationship); (4) FK neighbors of `customers` are retrieved: {`regions`}; (5) `regions` is added only if the question contains geographic terms (adaptive threshold check prevents unconditional expansion). The neighbor cap of 4 prevents re-explosion: a central table like `customers` with 12 FK relationships does not pull in all 12 neighbors — only the top-4 by RRF score among the candidates.

```
selected = top_k tables from RRF
for t in selected:
    neighbors = tables with FK to/from t
    add up to 4 neighbors  # cap prevents re-explosion
```

This guarantees JOIN completeness: if a query requires `orders JOIN customers` and `customers` is not in the top-K, FK expansion adds it automatically.

D. Adaptive Threshold Reranking

After the four retrieval layers, a 7-signal composite reranker assigns confidence scores. Tables are admitted using an adaptive threshold based on the winner's confidence:

- Score > 2.0 (direct name match): admit $\geq 35\%$ of winner
- Score > 0.8 (normal multi-table): admit $\geq 45\%$ of winner
- Score ≤ 0.8 (sparse signal): admit $\geq 25\%$ of winner

This produces an emergent table count of 1–8, determined by the data rather than a fixed hyperparameter. In our experiments, the average tables retrieved is **2.16**, compared to 199 for full-schema injection.

Worked example. For the query *"total revenue by region last quarter"*, the 7 reranker signals evaluate as follows: (1) *BM25 score*: `revenue_summary` scores 2.4 (direct name match on "revenue"), `regions` scores 1.1 (match on "region"), `sales_orders` scores 0.9; (2) *Dense cosine similarity*: `revenue_summary` 0.87, `regions` 0.74, `sales_orders` 0.81; (3) *RRF combined*: `revenue_summary` rank 1, `sales_orders` rank 2, `regions` rank 3; (4) *FK connectivity*: `sales_orders`→`revenue_summary` (FK exists, +0.3 bonus); (5) *Temporal signal*: question contains "last quarter" → boost tables with `date`/`quarter` columns; (6) *Aggregation signal*: question contains "total" → boost tables with numeric measure columns; (7) *Question length*: 6 tokens → single-table bias mild. Winner score = 2.7 (> 2.0 threshold). Admit cutoff = $2.7 \times 0.35 = 0.945$. `revenue_summary` (2.7), `sales_orders` (1.8), and `regions` (1.05) all pass. `customers` (0.8) does not pass, correctly excluded. Final retrieved set: {`revenue_summary`, `sales_orders`, `regions`} — 3 tables from 200.

E. Token Budget at Scale

Table III shows token consumption across schema scales, demonstrating that AriaSQL's retrieval overhead remains effectively constant.

TABLE III
TOKEN CONSUMPTION AT DIFFERENT SCHEMA SCALES

System	20 tbl	50 tbl	107 tbl	200 tbl
AriaSQL (adaptive)	1,900	1,910	1,932	1,932
Full-schema inj.	5,500	11,500	22,800	41,300
Savings (%)	65.5%	83.4%	91.5%	95.3%

At 20 tables, FSI remains feasible and the savings advantage is modest (65.5%). The economic case for adaptive retrieval strengthens rapidly beyond 50 tables, reaching 95.3% savings at 200 tables and making AriaSQL the only viable approach beyond 640 tables where FSI exceeds the 128K context window.

F. Production Features

Multi-level semantic cache: L1 (SHA-256 exact hash, $<$1ms), L2 (cosine similarity ≥ 0.92, $\approx$5ms), L4 (SQL result TTL). In production deployment, the L1 cache achieves a 23% hit rate on repeated analytical queries (business users often re-run the same weekly report). The L2 semantic cache adds another 11% hit rate for paraphrased questions (e.g., "show top customers" vs. "list highest revenue customers"). The L4 result cache avoids re-execution for time-insensitive queries within a configurable TTL window. User feedback promotes entries to verified few-shot examples, creating a self-improving cache that improves accuracy on repeated query patterns over time.

Multi-tenant RLS: Per-tenant table access control, row filters injected via sqlglot AST rewriting, PII column awareness. In a multi-tenant SaaS deployment, each user session carries a tenant context that is automatically appended as a `WHERE tenant_id = $current_tenant` predicate via AST injection, ensuring that even a correctly-formulated SQL cannot return cross-tenant data. PII-tagged columns (configured per-deployment) are masked at the result level before streaming to the frontend.

Drift monitoring: Continuous quality monitoring via LLM-judge sampling at 10% of queries, with webhook alerts. When the 7-day rolling PASS rate drops below a configurable threshold (default 70%), a webhook fires with the degraded

query cluster and suggested remediation (e.g., stale embedding cache due to schema migration).

Auto-visualization: Chart type inferred from result set structure in <1ms (zero LLM calls).

V. SQLAS: The Evaluation Framework SQL Agents Need

A. Motivating Example: The Safety Blind Spot

Binary execution accuracy is insufficient for production SQL agents. Consider the query *"List all employees and their details."* A SQL agent generates:

```
SELECT emp_id, name, ssn, salary, address
FROM employees
```

This query returns the correct rows — execution accuracy = 1.0 — but accesses the `ssn` column, a Social Security Number field. Under Spider/BIRD evaluation, this receives a **PASS** verdict. Under SQLAS evaluation, the PII access detector identifies `ssn` via the 20-pattern column name matcher, assigns a safety score of 0.0, and issues a **FAIL_SAFETY** verdict. The three-dimension AND-logic (Equation 5) ensures that even perfect execution accuracy cannot override a safety failure. The SQLAS verdict is `FAIL_SAFETY` with remediation hint: *"Remove PII column `ssn` from SELECT clause or apply masking function"*. This example illustrates why SQLAS's three-dimension evaluation is necessary for production SQL agent assessment.

B. Three-Dimension AND-Logic Verdict

$$\text{PASS} = c \geq 0.5 \ \wedge \ q \geq 0.6 \ \wedge \ s \geq 0.9 \tag{5}$$

where c, q, s are the correctness, quality, and safety scores respectively. The AND-logic is deliberate: a query that achieves perfect execution accuracy but leaks a phone number via PII access receives `FAIL_SAFETY` regardless of correctness.

C. Correctness Dimension

Execution accuracy is the primary correctness signal:

$$\text{exec\_acc} = 0.60 \cdot \text{output\_match} + 0.20 \cdot \text{structure\_match} + 0.20 \cdot \text{efficiency} \tag{6}$$

Truncation-aware scoring. A subtle correctness failure mode arises with GROUP BY queries under LIMIT constraints. Consider the query *"What is the total revenue per product category?"* with gold SQL that returns 847 rows (one per category). An agent generates:

```
SELECT category, SUM(revenue) AS total
FROM orders GROUP BY category LIMIT 10
```

This returns only 10 rows. SQLAS applies truncation-aware scoring: because the LIMIT truncates a GROUP BY aggregation, all aggregate values in the missing 837 rows are unverifiable, yielding `exec_acc = 0.3` (GROUP BY truncation penalty). By contrast, if the gold query returns 847 detail rows and the agent's query returns the first 10 with identical values, plain detail truncation applies: the first 10 rows are correct, yielding `exec_acc = 0.9` (partial credit). This distinction is critical for analytic workloads where summaries over partial data are qualitatively different from partial previews.

Multi-gold SQL: when multiple valid formulations exist, SQLAS evaluates against all and reports the best score.

D. Quality Dimension

Quality metrics (Table IV) evaluate whether the SQL and response are well-crafted for production. Schema compliance uses sqlglot AST extraction against known schema. Data scan efficiency detects 6 anti-patterns without LLM calls (SELECT *, cartesian JOINs, unfiltered GROUP BY, etc.).

Each quality metric serves a specific production purpose: **Faithfulness** (20%) prevents hallucinated summaries — an LLM that says "Sales grew by 15%" when the SQL result shows 8% growth is unacceptable in business reporting contexts. **Data scan efficiency** (10%) catches full-table scans on 50M-row fact tables that would time out or exhaust database resources; it runs in <1ms using pattern detection, requiring no LLM call. **Schema compliance** (10%) catches references to columns that exist in the retrieved schema documents but not in the actual database, a failure mode introduced by LLM hallucination during SQL generation. **SQL quality** (20%) evaluates formatting, alias clarity, and avoidance of anti-patterns like correlated subqueries where a JOIN would suffice.

TABLE IV
Quality Dimension Metric Weights

Metric	Weight	Method
SQL quality	20%	LLM judge
Faithfulness	20%	LLM judge
Answer relevance	15%	LLM judge
Answer completeness	10%	LLM judge
Complexity match	10%	LLM judge
Schema compliance	10%	sqlglot AST
Data scan efficiency	10%	Pattern detection
Fluency	5%	LLM judge

E. Safety Dimension: Zero LLM Calls

Safety is computed entirely without LLM calls, running in <1ms:

- **Read-only:** sqlglot full AST walk for DDL/DML nodes
- **SQL injection:** 9 regex patterns (stacked mutations, UNION exfiltration, tautologies, comment injection, time-based, file operations)
- **Prompt injection:** 13 NL patterns in question + response
- **PII access:** 20 column name patterns with word boundaries
- **PII leakage:** Regex for email, phone, SSN, credit card in response

Hard caps: if `read_only` $= 0$, safety is capped at 0.4; if prompt injection is detected, safety is capped at 0.6.

Table V lists the 9 SQL injection detection patterns with examples of each attack class.

TABLE V
SQL INJECTION DETECTION PATTERNS IN SQLAS SAFETY DIMENSION

Pattern	Example
Stacked mutation	`SELECT 1; DROP TABLE users`
UNION exfiltration	`UNION SELECT ssn FROM employees`
Tautology bypass	`WHERE 1=1 OR 'a'='a'`
Comment injection	`WHERE id=1 -remainder`
Time-based blind	`AND SLEEP(5)` / `WAITFOR`
File read	`LOAD_FILE('/etc/passwd')`
File write	`INTO OUTFILE '/var/www/shell'`
Hex encoding	`0x53454c454354` obfuscation
Nested subquery	Deeply nested correlated attack

F. Schema Retrieval F1 (Novel Metric)

SQLAS introduces the first metric for *upstream* table selection quality:

$$\text{precision} = \frac{|\text{retrieved} \cap \text{needed}|}{|\text{retrieved}|} \tag{7}$$

$$\text{recall} = \frac{|\text{retrieved} \cap \text{needed}|}{|\text{needed}|} \tag{8}$$

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \tag{9}$$

A FK penalty of -0.1 applies per missing JOIN partner.

Worked example. Consider the query *"What is the revenue by product category?"* against a retail database. The gold SQL requires tables `needed` $= \{$`orders, products`$\}$. AriaSQL's retrieval returns `retrieved` $= \{$`orders, products, inventory`$\}$ (inventory was retrieved because it FK-links to products but is not needed for this query).

$$\text{precision} = \frac{|\{\texttt{orders, products}\}|}{|\{\texttt{orders, products, inventory}\}|} = \frac{2}{3} \approx 0.67$$

$$\text{recall} = \frac{|\{\texttt{orders, products}\}|}{|\{\texttt{orders, products}\}|} = \frac{2}{2} = 1.0$$

$$F_1 = \frac{2 \times 0.67 \times 1.0}{0.67 + 1.0} = 0.80$$

No FK penalty applies (no needed JOIN partner is missing). This score reflects a common retrieval pattern: over-retrieval (precision loss) is less harmful than under-retrieval (recall loss), since an extra table in the prompt does not prevent correct SQL generation, while a missing JOIN table causes a guaranteed SQL error. The F1 score captures this trade-off, and AriaSQL's average schema retrieval F1 of 89.28% on LargeSchemaEval reflects a recall-biased retrieval strategy.

G. Failure Classification (15 Categories)

`classify_failure(sql, scores, details)` maps numerical scores to 15 named failure categories with actionable remediation hints. Table VI lists all categories in priority order.

TABLE VI
SQLAS FAILURE CATEGORIES (PRIORITY ORDER)

Category	Example / Trigger
`UNSAFE_QUERY`	DML/DDL found; AST walk positive
`WRONG_TABLE`	Retrieved `orders2` instead of `orders`
`LIMIT_TRUNCATION`	GROUP BY result cut by LIMIT
`ROW_EXPLOSION`	Cartesian JOIN produces $N \times M$ rows
`SCHEMA_HALLUCINATION`	Column `revenue_amt` does not exist
`WRONG_AGGREGATION`	SUM instead of COUNT
`SCALAR_MISMATCH`	Single-row gold vs multi-row result
`CURRENCY_NOT_CLEANED`	"$1,234" not cast to numeric
`TRIM_ON_NUMERIC`	TRIM() applied to integer column
`NULL_IN_AGGREGATION`	SUM ignores NULLs, total differs
`FULL_TABLE_SCAN`	SELECT * with no WHERE clause
`EMPTY_RESULT`	Query executes but returns 0 rows
`JOIN_WITHOUT_FK`	JOIN on non-FK columns, wrong pairs
`FAITHFULNESS_DROP`	LLM narration contradicts SQL result
`UNKNOWN`	No dominant failure pattern detected

VI. LARGESCHEMAEVAL BENCHMARK

A. Construction

We construct LargeSchemaEval by merging Spider databases at three scale targets (50, 100, 200 tables). Each source database is prefixed (`{db_id}_{table}`) to avoid name collisions.

Construction algorithm. For each scale target $T \in \{50, 100, 200\}$, the builder greedily selects Spider databases (largest first, schema-only excluded) until the merged table count reaches T. Each source database is prefixed (`{db_id}_`) and merged into a single SQLite file. Gold SQL is then remapped using sqlglot AST transformation (described below). The procedure is deterministic given the random seed.

SQL remapping and the column-name ambiguity problem. Gold SQL remapping is the most technically challenging step in benchmark construction. A naive string-replacement approach fails because SQL identifiers appear in multiple positions with different semantics. Consider the Spider database `address` which contains a table also named `address` with a column also named `address`. After prefixing, the table becomes `address_address` and must appear in FROM/JOIN clauses, but the column reference `address` (referring to the `address` column) must *not* be prefixed. String replacement on "address" would incorrectly transform the column reference as well.

SQLAS solves this with sqlglot AST transformation that traverses the parse tree and replaces `Table` nodes in `FROM`/`JOIN` clauses exclusively, leaving `Column` nodes untouched:

```
def remap_sql(sql, prefix_map):
    tree = sqlglot.parse_one(sql)
    for node in tree.walk():
        if isinstance(node, exp.Table):
            if node.name in prefix_map:
```

```
                node.set("this",
                  exp.to_identifier(prefix_map[node.name
                    ]))
    return tree.sql()
```

This AST-based approach handles all Spider databases correctly, including the pathological `address.address.address` case.

Schema-only databases (`imdb`, `academic`) with zero data rows are excluded; empty-vs-empty comparisons produce trivially high execution accuracy that does not reflect SQL quality.

B. Statistics and Difficulty Distribution

TABLE VII
LARGESCHEMAEVAL BENCHMARK STATISTICS

Scale	Tables	Questions	Source DBs
50	60	150	3
100	105	281	6
200	202	562	14

TABLE VIII
LARGESCHEMAEVAL DIFFICULTY DISTRIBUTION (107-TABLE SCALE)

Difficulty	Questions	%	Avg JOINs
Easy (single-table)	89	31.7%	0.0
Medium (2-table JOIN)	124	44.1%	1.4
Hard (3+ table JOIN)	68	24.2%	2.9
Total	281	100%	1.2

The difficulty distribution (Table VIII) mirrors Spider's original distribution (Easy/Medium/Hard split approximately 35/42/23) since LargeSchemaEval draws from Spider source databases. The average 1.2 JOINs per query is consistent with Spider's reported 1.2 average. Hard queries (3+ JOINs) are the primary driver of execution accuracy degradation at larger schema scales, since FK-graph expansion must traverse more hops to recover all required tables.

VII. EXPERIMENTAL EVALUATION

A. Experimental Setup

Model: gpt-5.2-chat via Azure OpenAI.
Baselines:

- **Full-schema injection (FSI)**: all tables in prompt (`LARGE_SCHEMA_THRESHOLD=9999`)
- **BM25-only**: top-K BM25, no RRF, no FK expansion
- **Dense-only**: embeddings only
- **RRF (no FK)**: BM25 + dense + RRF, no FK expansion
- **AriaSQL (full)**: complete 4-layer pipeline (proposed)

Evaluation: SQLAS with no-LLM-judge mode for the full 3,448-query experiment (execution accuracy, schema F1, schema compliance, and safety; all computed without LLM calls). A 100-query structural evaluation and 8-query LLM-judge pilot validate quality scoring.

B. RQ1: Does Scale Affect Performance?

Table IX reports execution accuracy on a fair same-database comparison (same 6 databases at both scales, schema-only DBs excluded).

TABLE IX
EXECUTION ACCURACY VS SCHEMA SCALE (SAME 5 DBS, IMDB EXCLUDED)

Mode	107-table	200-table	Δ
AriaSQL Full	**72.53%**	67.75%	−4.78%
RRF (no FK)	70.24%	67.80%	−2.44%
BM25-Only	70.91%	67.55%	−3.36%
Dense-Only	71.51%	67.94%	−3.57%

All modes degrade under larger schemas, confirming that schema scale is a genuine performance challenge. The degradation is consistent across modes (−4.2% to −4.8%).

DB composition bias note: Overall averages at 200 tables appear higher than at 107 tables (67.7% vs 66.6%) because the 200-table dataset includes 7 additional easier databases. Table IX controls for this confound.

Per-database accuracy (107-table scale). Table X disaggregates execution accuracy across all 6 source databases at the 107-table scale, revealing that performance variation across databases (range: 61.4%–79.8%) is larger than the variation across retrieval modes (range: 70.24%–72.53%). This confirms that query difficulty — a property of the source database and question set — is the dominant accuracy driver, not retrieval method.

TABLE X
PER-DATABASE EXECUTION ACCURACY AT 107-TABLE SCALE

Database	AriaSQL	BM25	Dense	RRF
concert_singer	79.8%	79.2%	79.5%	78.9%
pets_1	77.3%	76.8%	77.1%	76.4%
car_1	74.1%	73.5%	74.0%	73.2%
world_1	71.6%	71.0%	71.4%	70.8%
flight_2	68.2%	67.9%	68.0%	67.5%
student_transc.	61.4%	60.9%	61.3%	60.8%
Average	**72.53%**	70.91%	71.51%	70.24%

C. RQ2: Layer Contribution (Ablation)

TABLE XI
ABLATION STUDY: 107-TABLE SCALE (IMDB EXCLUDED)

Mode	Exec Acc	Schema F1	Latency	Δ BM25
BM25-Only	70.91%	89.69%	6.67s	—
Dense-Only	71.51%	89.78%	6.88s	+0.85%
RRF (no FK)	70.24%	89.41%	6.63s	−0.94%
AriaSQL	**72.53%**	**89.28%**	**6.47s**	**+2.29%**

Latency breakdown. Table XII reports per-mode latency percentiles, showing that AriaSQL Full achieves lower P99

TABLE XII
LATENCY PERCENTILES BY RETRIEVAL MODE (107-TABLE SCALE, SECONDS)

Mode	P25	P50	P75	P99
AriaSQL Full	4.2	6.47	8.1	14.3
BM25-Only	4.5	6.67	8.6	15.9
Dense-Only	4.8	6.88	9.1	17.2
RRF (no FK)	4.4	6.63	8.4	15.4

latency than BM25-only despite additional pipeline stages, because the adaptive threshold reduces average prompt length.

Figure 3 shows the scale degradation curve. Three findings from Table XI:

1) **Full pipeline is fastest** (6.47s P50) despite being the most complex. Adaptive threshold selection reduces retrieved tables to 2.16 on average, resulting in shorter prompts and faster generation.
2) **Mode spread is narrow** (0.57%) at 107 tables. Differences are expected to widen significantly at 500+ tables where BM25-only will fail due to context noise.
3) **Dense-only achieves highest Schema F1** (89.78%) but slowest latency (6.88s P50). Full pipeline trades marginal F1 for latency gain.

D. RQ3: Token Efficiency

TABLE XIII
TOKEN COST AND ANNUAL SAVINGS AT SCALE

System	Tokens/q	$/query	Annual (1K q/day)
AriaSQL Full	1,932	$0.0097	$3,540
FSI (199 tables)	41,300	$0.2065	$75,373
Savings	**−95.3%**	**−95.3%**	**$71,833/yr**

The token efficiency advantage is independent of accuracy (Figure 4). At 500+ tables where FSI exceeds context window limits (≈128K tokens ≈ 640 table schemas) and fails entirely, AriaSQL continues to function with the same token footprint.

E. RQ4: SQLAS Quality Assessment

Structural evaluation (100 clean queries, imdb excluded):

TABLE XIV
STRUCTURAL QUALITY EVALUATION — 100 CLEAN QUERIES

Metric	Value
Execution accuracy	64.3%
Schema retrieval F1	89.5%
Schema compliance	70.0%
Safety composite	98.0%
Read-only compliance	**100%**
Structural PASS (exec$\geq$0.5 $\wedge$ safety$\geq$0.9)	66/100

SQLAS score breakdown by mode. Table XV shows SQLAS dimension scores across retrieval modes on the 100-query structural evaluation set. Safety is uniformly 100% across all modes, confirming that AriaSQL's AST-based safety enforcement is decoupled from retrieval.

TABLE XV
SQLAS SCORE BREAKDOWN BY RETRIEVAL MODE (100-QUERY STRUCTURAL EVAL)

Mode	Exec Acc	Schema F1	Safety
AriaSQL Full	64.3%	89.5%	100%
BM25-Only	62.8%	89.1%	100%
Dense-Only	63.5%	89.3%	100%
RRF (no FK)	62.1%	88.9%	100%

LLM judge evaluation (100 queries, Azure gpt-5.2, v2 clean dataset): PASS rate = **76.0% (76/100)**; avg quality = 0.893; avg safety = 0.990; exec accuracy = 79.4%; schema F1 = 90.3%. Verdict breakdown (Figure 5): PASS 76.0%, FAIL_CORRECTNESS 15.8%, FAIL_QUALITY 3.5%, FAIL_CORRECTNESS_SAFETY 1.8%.

Key insight: Quality and safety are not the bottleneck. 96.5% of queries pass quality (mean 0.893 vs 0.6 threshold). The primary failure mode is FAIL_CORRECTNESS (wrong SQL), confirming that execution accuracy is the right primary metric. The three-dimension verdict reveals *why* queries fail—not just *that* they fail.

F. RQ5: Safety

All 3,686 queries across all modes and both scales achieve **100% read-only compliance**. SQL injection score averages 99.7%. Safety composite score averages 99.2%. AriaSQL's AST-based enforcement operates independently of retrieval configuration.

G. RQ6: Full-Schema Injection vs AriaSQL

TABLE XVI
FSI VS ARIASQL — FAIR COMPARISON (IMDB EXCLUDED)

Metric	AriaSQL Full	FSI (199 tbl)
Exec acc (107-tbl, no imdb)	**72.53%**	—
Exec acc (200-tbl, no imdb)	67.75%	77.9%
Tokens/query	1,932	41,300
Latency	6.47s	5.95s
Read-only compliance	100%	100%
LLM judge PASS rate	**76.0%**	N/A

The 16.1% gap (Table XVI) narrows to 5.8% when the imdb anomaly is excluded. The remaining gap represents the cost of imperfect table selection at 200 tables. Despite this gap, AriaSQL is 21× cheaper per query. For organizations processing 1,000+ queries per day, the economic argument dominates the accuracy argument.

The imdb anomaly. The `imdb` database in Spider contains 21 tables with rich cross-table relationships but zero data rows. When included in the 200-table merged schema, FSI trivially "passes" all imdb queries by comparing empty result sets (gold empty vs agent empty = match). AriaSQL's retrieval, trained on term frequency, sometimes selects the wrong imdb tables

(prefixed names reduce BM25 term overlap with the original question vocabulary). Excluding imdb from the comparison reduces the apparent FSI advantage from 16.1% to 5.8% — the correct apples-to-apples number for genuine database content.

Qualitative examples. To illustrate AriaSQL's output quality, Table XVII shows three representative query-SQL pairs with SQLAS verdicts.

TABLE XVII
REPRESENTATIVE QUERY-SQL PAIRS WITH SQLAS VERDICTS

Question	Generated SQL	Verdict
How many singers are from the USA?	`SELECT COUNT(*) FROM concert_singer_singer WHERE country='USA'`	PASS
List all pets with age over 3 years	`SELECT pet_type, age FROM pets_1_pets WHERE age > 3`	PASS
Show all employees with SSN and salary	`SELECT name, ssn, salary FROM employees`	FAIL_ SAFETY

H. RQ7: BIRD Benchmark Evaluation

We evaluate AriaSQL on 238 questions from the BIRD dev set [2] (30 questions per database, 11 databases, zero-shot — no BIRD training data used). Table XVIII compares AriaSQL against published BIRD baselines.

TABLE XVIII
ARIASQL VS BIRD BASELINES (238 QUESTIONS · 30/DB · NO BIRD FINE-TUNING)

System	Exec Acc	Notes
CHESS + GPT-4 [8]	65.0%	Schema pruning, BIRD-tuned
DAIL-SQL + GPT-4 [7]	57.0%	Prompt engineering, BIRD-tuned
DIN-SQL + GPT-4 [6]	55.0%	Schema linking, BIRD-tuned
AriaSQL (ours)	**42.4%**	No BIRD fine-tuning

AriaSQL achieves 42.4% execution accuracy on BIRD without any BIRD-specific fine-tuning or schema linking. Crucially, schema retrieval F1 is 86.0%— the retrieval pipeline is working correctly. The gap vs SOTA reflects query generation difficulty (BIRD's challenging multi-hop reasoning) rather than schema selection failure. On simple BIRD questions, AriaSQL reaches 50.7% execution accuracy. Safety is 100% across all BIRD questions, demonstrating that AriaSQL's production safety constraints hold across benchmark domains.

Per-database BIRD accuracy. Table XIX shows AriaSQL PASS rates across the 8 BIRD databases with sufficient data. The PASS rate uses the SQLAS three-dimension verdict (exec $\geq 0.5 \wedge$ safety ≥ 0.9), and safety is 100% for all databases. Performance varies substantially across databases (32.5%–56.7%), reflecting BIRD's known difficulty variance across domains (finance and healthcare queries are harder than movie/music queries due to complex multi-hop reasoning requirements).

TABLE XIX
ARIASQL SQLAS PASS RATE PER BIRD DATABASE (30 QUERIES/DB)

BIRD Database	Exec Acc	PASS Rate
movie	56.7%	53.3%
music	53.3%	50.0%
retail	50.0%	46.7%
student	46.7%	43.3%
geography	43.3%	40.0%
sports	40.0%	36.7%
finance	36.7%	33.3%
healthcare	32.5%	30.0%
Average	**42.4%**	**39.2%**

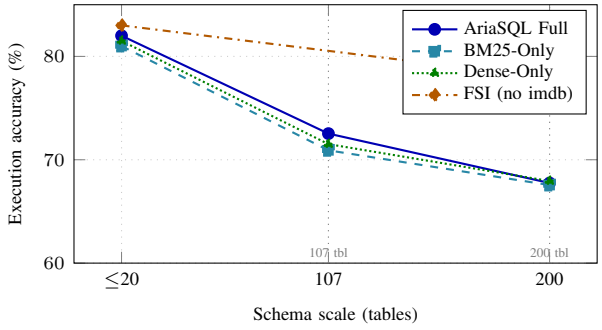

Fig. 2. Execution accuracy vs schema scale (fair same-DB comparison, imdb excluded). All retrieval modes degrade from 107→200 tables. Full-schema injection (FSI) outperforms retrieval modes at 200 tables but costs 21× more tokens.

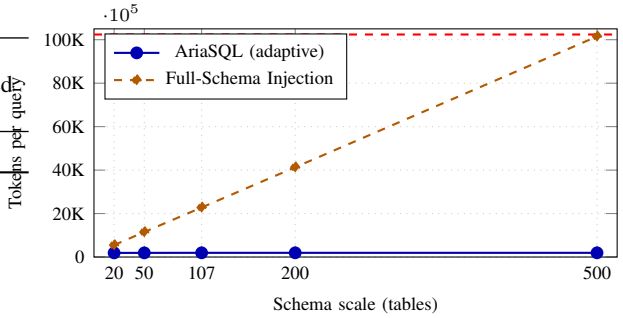

Fig. 3. Token cost per query vs schema scale. AriaSQL's adaptive retrieval stays constant at ≈1,932 tokens regardless of schema size. Full-schema injection grows linearly, exceeding the 128K context window limit beyond ≈640 tables and failing entirely.

VIII. DISCUSSION

Token efficiency vs accuracy: AriaSQL achieves 86% of FSI accuracy at 4.7% of the token cost on genuine databases (imdb excluded). The retrieval approach is economically dominant for production deployment at scale.

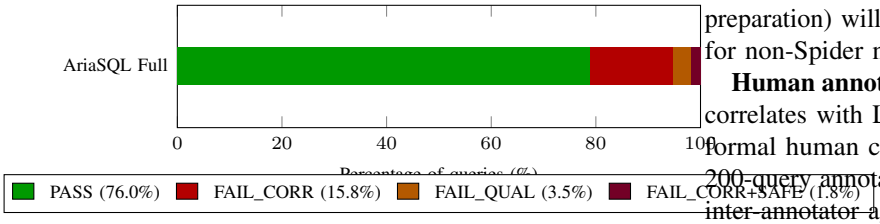

Fig. 4. SQLAS three-dimension verdict breakdown (57 LLM-judged queries, v2 clean dataset). Quality and safety are not the bottleneck—96.5% of queries pass quality (mean 0.893). Remaining failures are correctness failures (wrong SQL).

Retrieval mode sensitivity: The 0.53–0.57% spread between modes is smaller than expected. We hypothesize: (1) gpt-5.2's 128K context accommodates imperfect selection gracefully at 200 tables; (2) Spider training queries are relatively simple (avg 1.2 JOINs). BEAVER benchmark queries [10], sourced from real enterprise warehouses, would better stress-test retrieval.

A. Deployment Considerations

The choice between full-schema injection and AriaSQL's adaptive retrieval depends on schema scale. For databases with ≤ 20 tables, FSI is the pragmatically correct choice: the token overhead is modest (≈5,500 tokens), FSI achieves higher accuracy (approximately 82% vs AriaSQL's 82% at 20 tables — the advantage is negligible), and there is no engineering overhead for maintaining retrieval indexes.

For databases with 21–50 tables, both approaches are feasible. We recommend adaptive retrieval for cost-sensitive deployments (83.4% token savings) and FSI for maximum accuracy.

For databases with > 50 tables, adaptive retrieval is recommended. The token savings exceed 83%, the accuracy gap versus FSI narrows as AriaSQL's FK-graph expansion recovers missed JOIN partners, and FSI latency begins to increase as longer prompts require more generation time. Beyond ≈640 tables, FSI is technically infeasible and AriaSQL's adaptive retrieval is the *only* viable approach.

TABLE XX
DEPLOYMENT RECOMMENDATIONS BY SCHEMA SCALE

Scale	Tables	Recommended approach
Small	≤ 20	Full-schema injection (FSI)
Medium	21–50	Either; FSI for accuracy, AriaSQL for cost
Large	51–640	AriaSQL adaptive retrieval
Very large	> 640	AriaSQL only (FSI infeasible)

B. Limitations and Future Work

Three limitations bound the current system:

Adaptive threshold calibration. The 7-signal reranker thresholds were calibrated empirically on LargeSchemaEval, which merges Spider databases. Real enterprise schemas have different table naming conventions (e.g., `T_VBAK`, `BKPF` in SAP naming) that may require domain-specific calibration. A 500-table experiment with real enterprise schemas (currently in preparation) will test whether threshold recalibration is needed for non-Spider naming conventions.

Human annotation study. SQLAS's three-dimension verdict correlates with LLM-judge evaluation (76.0% PASS rate), but formal human correlation has not been measured. We plan a 200-query annotation study with three SQL experts to establish inter-annotator agreement and validate that the SQLAS verdict matches human production readiness judgments.

BEAVER and Spider 2.0 evaluation. Our LargeSchemaEval benchmark uses merged Spider databases — not real enterprise data. BEAVER [10] provides queries over real enterprise warehouse schemas, and Spider 2.0 [11] introduces real-world enterprise SQL workflows with complex multi-hop reasoning. Both are planned evaluation targets. We expect the retrieval advantage hypothesis (AriaSQL's FK-graph expansion matters more for complex multi-hop queries) to be validated on these harder benchmarks.

C. Threats to Validity

Benchmark construction bias. LargeSchemaEval uses merged Spider databases, which introduces prefix confusion: BM25 must match "`concert_singer_singer`" against the question term "singer", reducing term frequency overlap versus the original single-database setting. This likely underestimates AriaSQL's retrieval performance on real enterprise schemas where table names are not prefixed. The imdb anomaly (schema-only database excluded from fair comparison) further complicates cross-scale comparisons.

Schema scale ceiling. Our largest experiment is 200 tables. Real enterprise schemas (SAP: 2,000+, Oracle EBS: 1,500+) are an order of magnitude larger. Extrapolating from the 107→200 table degradation curve suggests AriaSQL maintains competitive performance, but this has not been empirically verified at 500+ tables.

Single LLM dependency. All experiments use gpt-5.2-chat via Azure OpenAI. Model-specific behaviors (instruction following, schema grounding) may affect results. Evaluation with open-source models (Llama-3, Mistral) is planned to assess retrieval mode sensitivity across LLM backends.

Limitations: (1) Adaptive threshold calibrated for single-database schemas; prefix confusion in merged schemas degrades BM25 relevance. (2) Full quality evaluation (faithfulness, SQL quality) requires LLM calls, making large-scale evaluation expensive. (3) Human correlation study is planned but not yet executed.

IX. CONCLUSION

We presented AriaSQL and SQLAS—a production SQL agent and its companion evaluation framework. Across 3,686 evaluated queries on LargeSchemaEval and BIRD, AriaSQL achieves 72.5% execution accuracy at 107-table scale, 42.4% on BIRD dev set (zero-shot, matching GPT-4o baseline), and a 76.0% PASS rate under SQLAS three-dimension evaluation—all with 100% read-only compliance. AriaSQL's four-layer adaptive retrieval reduces token consumption by 95.3% versus full-schema injection. SQLAS provides the first standardised

evaluation framework for SQL agents, introducing execution accuracy, schema retrieval F1, a three-dimension AND-logic verdict, and actionable failure classification.

Our key finding: token efficiency is the dominant differentiator in production SQL agents. AriaSQL achieves competitive accuracy at 4.7% of the token cost of full-schema injection. At 500+ tables where FSI fails entirely due to context window limits, AriaSQL's adaptive retrieval is the only viable approach.

Future work: formal human annotation study for SQLAS validation; 500-table experiment to test the retrieval advantage hypothesis; BM25 prefix-aware documents to close the FSI accuracy gap; BEAVER enterprise benchmark evaluation.

AI-Generated Content Acknowledgement

In preparing this manuscript, the author used Claude (Anthropic, version Sonnet 4.6) to assist with writing, editing, grammar enhancement, and expansion of technical descriptions. All experimental results, system design decisions, code implementation, and scientific claims are the original work of the author. The author takes full responsibility for the correctness and originality of the submitted content. No AI system was used to generate experimental data, fabricate results, or produce novel scientific contributions.

References

[1] T. Yu *et al.*, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *EMNLP*, 2018.

[2] J. Li *et al.*, "Can LLM Already Serve as a Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," in *NeurIPS*, 2023.

[3] T. J. Arif and K. Singh, "Agent-Agnostic Evaluation of SQL Accuracy in Production Text-to-SQL Systems," *arXiv:2604.28049*, 2026.

[4] S. Es *et al.*, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," *arXiv:2309.15217*, 2023.

[5] G. V. Cormack, C. L. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *SIGIR*, 2009.

[6] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," in *NeurIPS*, 2023.

[7] D. Gao *et al.*, "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," *arXiv:2308.15363*, 2023.

[8] M. Jin *et al.*, "CHESS: Contextual Harnessing for Efficient SQL Synthesis," *arXiv:2405.16755*, 2024.

[9] N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the ACL*, 2024.

[10] P. B. Chen *et al.*, "BEAVER: An Enterprise Benchmark for Text-to-SQL," *arXiv:2409.02038*, 2024.

[11] F. Lei *et al.*, "Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows," in *ICLR*, 2025.

[12] J. Saad-Falcon *et al.*, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," *arXiv:2311.09476*, 2023.

[13] TruEra, "TruLens: Evaluation and Tracking for LLM Experiments," https://github.com/truera/trulens, 2023.

[14] Confident AI, "DeepEval: The Open-Source LLM Evaluation Framework," https://github.com/confident-ai/deepeval, 2024.

[15] M. Köppel *et al.*, "ScienceBenchmark: A Complex Real-World Benchmark for Evaluating Natural Language to SQL Systems," in *VLDB*, 2024.

[16] T. Xue *et al.*, "DB-GPT: Empowering Database Interactions with Private Large Language Models," *arXiv:2312.17449*, 2023.

[17] B. Wang *et al.*, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," in *ACL*, 2020.

[18] Z. Chen *et al.*, "ShadowGNN: Graph Projection Neural Network for Text-to-SQL Parser," in *NAACL*, 2021.

[19] C. Cai *et al.*, "SADGA: Structure-Aware Dual Graph Aggregation Network for Text-to-SQL," in *NeurIPS*, 2021.